

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – CODING CHALLENGE

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	Assessment:	Assessment Tool 3 – Coding Challenge
Weightage:	30% of CIA		
CO5 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	B.Harsha Vardhan	Reg. No.:	192425201
Section / Batch:	2024	Date:	10/07/26

Code of Conduct

I B.Harsha Vardhan(192425201) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: B.Harsha Vardhan_

Instructions

- Develop a complete and modular Python program that meets all the functional requirements specified in the coding challenge using appropriate algorithms, data structures, and programming practices.
- Test the program using multiple valid and invalid test cases, and clearly display the input, processing, and output to verify the correctness of the implementation.
- Follow standard coding practices by using meaningful variable names, proper indentation, comments, exception handling (where applicable), and upload the source code, sample outputs, and README file to the designated GitHub repository.
- Duration: 40 minutes.

Section / Topic	Focus	Q Nos
Regular Expression Pattern Matching	Regular Expressions, Basic Regular Expression Patterns	1
Finite State Automata Implementation	Finite State Automata, Models and Algorithms	2
Advanced Regular Expression Search	Regular Expressions, Basic Regular Expression Patterns	3

Section 1: Intelligent Email and Password Validator using Regular Expressions

1. Problem Statement:

A software company is developing a user registration system. Before creating an account, the system must validate user credentials based on predefined security policies.

Develop a Python application that validates:

Email Address

Password

Mobile Number

using Regular Expressions.

Validation Rules

Email

Must start with an alphabet.

May contain letters, digits, '.', '_' before '@'.

Domain should contain only alphabets.

Valid extensions: .com, .org, .edu, .net, .in

Password

Minimum 8 characters

At least one uppercase letter

At least one lowercase letter

At least one digit

At least one special character (@, #, \$, %, &, !)

Mobile Number

Must contain exactly 10 digits.

First digit should be between 6–9.

Expected Output

Display:

Valid Email / Invalid Email

Strong Password / Weak Password

Valid Mobile Number / Invalid Mobile Number

CODE:

```
import re
```

```
# ----- Regex patterns -----
```

```
EMAIL = re.compile(r"^[A-Za-z][A-Za-z0-9._]*@[A-Za-z]+\.(?:com|org|edu|net|in)$")
```

```
PASSWORD = re.compile(
```

```
    r"^(?=.*[A-Z])"           # at least one uppercase
```

```
    r"(?=.*[a-z])"           # at least one lowercase
```

```
    r"(?=.*\d)"               # at least one digit
```

```
    r"(?=.*[@#$%&!])"         # at least one special character
```

```
    r".{8,}$"                 # minimum 8 characters
```

```
)
```

```
MOBILE = re.compile(r"^[6-9]\d{9}$")
```

```
# ----- Validation functions -----
```

```
def validate_email(email):
```

```
    return "Valid Email" if EMAIL.fullmatch(email) else "Invalid Email"
```

```

def validate_password(password):
    return "Strong Password" if PASSWORD.fullmatch(password) else "Weak Password"

def validate_mobile(mobile):
    return "Valid Mobile Number" if MOBILE.fullmatch(mobile) else "Invalid Mobile Number"

# ----- Main program -----
def main():
    email = input("Enter Email : ").strip()
    password = input("Enter Password : ").strip()
    mobile = input("Enter Mobile : ").strip()

    print("\n--- Validation Result ---")
    print("Email :", validate_email(email))
    print("Password :", validate_password(password))
    print("Mobile :", validate_mobile(mobile))

    if (EMAIL.fullmatch(email) and PASSWORD.fullmatch(password)
        and MOBILE.fullmatch(mobile)):
        print("\nRegistration Successful")
    else:
        print("\nRegistration Failed")

if __name__ == "__main__":
    main()

```

OUTPUT:

```

PS D:\PYTHON VS> & C:\Users\badri\AppData\Local\Programs\Python\Python314\python.exe "d:/PYTHON VS/1-python basics/CO1-AT3-NLP-EXP1.py"
Enter Email : harsha@gmail.com
Enter Password : AHS1#fd@45
Enter Mobile : 2314456789

--- Validation Result ---
Email : Valid Email
Password : Strong Password
Mobile : Invalid Mobile Number

Registration Failed
PS D:\PYTHON VS> & C:\Users\badri\AppData\Local\Programs\Python\Python314\python.exe "d:/PYTHON VS/1-python basics/CO1-AT3-NLP-EXP1.py"
Enter Email : anita@company.in
Enter Password : Abcdefi!
Enter Mobile : 7012345678

--- Validation Result ---
Email : Valid Email
Password : Strong Password
Mobile : Valid Mobile Number

Registration Successful
PS D:\PYTHON VS>

```

2. Problem Statement

Develop a Python-based Deterministic Finite Automata (DFA) simulator.

The program should:

- Accept the DFA description
 - States
 - Input alphabet
 - Transition table
 - Initial state
 - Final state(s)
- Accept multiple input strings.
- Simulate state transitions.
- Display whether the string is Accepted or Rejected.

Example

Language:

Strings ending with "ab"

Input

abaab

Output

Transition Path:

$q_0 \rightarrow q_1 \rightarrow q_0 \rightarrow q_1 \rightarrow q_2$

Accepted

CODE:

class DFA:

```
def __init__(self, states, alphabet, transitions, start, finals):
```

```
    self.states = states
```

```
    self.alphabet = alphabet
```

```
    self.transitions = transitions
```

```
    self.start = start
```

```
    self.finals = finals
```

```
def run(self, string):
```

```
    """Return (path, accepted)."""
```

```
    current = self.start
```

```
    path = [current]
```

```
    for ch in string:
```

```
        if ch not in self.alphabet:
```

```
            return path, False          # invalid symbol
```

```
        current = self.transitions[current][ch]
```

```
        path.append(current)
```

```
    return path, current in self.finals
```

```
def default_dfa():
```

```
    return DFA(
```

```
        states=["q0", "q1", "q2"],
```

```
        alphabet=["a", "b"],
```

```
        transitions={
```

```
            "q0": {"a": "q1", "b": "q0"},
```

```
            "q1": {"a": "q1", "b": "q2"},
```

```
            "q2": {"a": "q1", "b": "q0"},
```

```

    },
    start="q0",
    finals=["q2"],
)

```

```

def read_dfa():
    states = input("States (comma separated) : ").replace(" ", "").split(",")
    alphabet = input("Alphabet (comma separated) : ").replace(" ", "").split(",")
    transitions = {}
    print("Enter transitions:")
    for s in states:
        transitions[s] = {}
        for a in alphabet:
            transitions[s][a] = input(f" delta({s}, {a}) = ").strip()
    start = input("Initial state : ").strip()
    finals = input("Final state(s) (comma sep.) : ").replace(" ", "").split(",")
    return DFA(states, alphabet, transitions, start, finals)

```

```

def main():
    choice = input("Use default DFA (strings ending with 'ab')? (y/n): ").lower()
    dfa = default_dfa() if choice == "y" else read_dfa()

```

```

    strings = input("Enter input strings (space separated): ").split()
    for s in strings:
        path, accepted = dfa.run(s)
        print(f"\nInput: {s if s else '(empty)'}")
        print("Transition Path:", " → ".join(path))
        print("Accepted" if accepted else "Rejected")

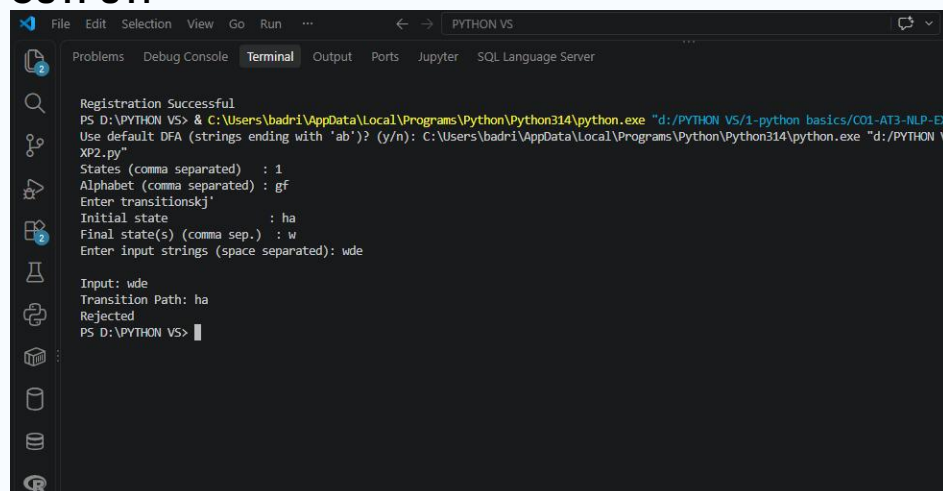
```

```

if __name__ == "__main__":
    main()

```

OUTPUT:



```

Registration Successful
PS D:\PYTHON VS> & C:\Users\badri\AppData\Local\Programs\Python\Python314\python.exe "d:/PYTHON VS/1-python basics/CO1-AT3-NLP-EXP
Use default DFA (strings ending with 'ab')? (y/n): C:\Users\badri\AppData\Local\Programs\Python\Python314\python.exe "d:/PYTHON VS
XP2.py"
States (comma separated) : 1
Alphabet (comma separated) : gf
Enter transitionskj'
Initial state : ha
Final state(s) (comma sep.) : w
Enter Input strings (space separated): wde

Input: wde
Transition Path: ha
Rejected
PS D:\PYTHON VS>

```

3. Problem Statement

Develop a Python-based text search engine that performs pattern matching using Regular Expressions.

The system should support:

Word Search
Prefix Search
Suffix Search
Date Search
Phone Number Search
Hashtag Search
Mention (@username) Search

Example Input

Meeting on 12/09/2026
Call 9876543210
#NLP
@OpenAI
natural language processing

User Menu

1.Search Date
2.Search Phone Number
3.Search Hashtag
4.Search Mention
5.Search Prefix
6.Search Suffix

Expected Output

Display all matching patterns based on user selection.

CODE:

```
import re
```

```
TEXT = """Meeting on 12/09/2026  
Call 9876543210  
#NLP  
@OpenAI  
natural language processing"""
```

```
# ----- Fixed patterns -----  
DATE = r"\b\d{1,2}[/-]\d{1,2}[/-]\d{4}\b"  
PHONE = r"\b[6-9]\d{9}\b"  
HASHTAG = r"#\w+"  
MENTION = r"@ \w+"
```

```
def find(pattern, text):  
    return [m.group() for m in re.finditer(pattern, text, re.I)]
```

```
def show(title, results):  
    print(f"\n{title}: {len(results)} match(es)")  
    for r in results:  
        print("  -", r)  
    if not results:  
        print("  (no match)")
```

```

MENU = ""
===== TEXT SEARCH ENGINE =====
1. Search Date
2. Search Phone Number
3. Search Hashtag
4. Search Mention
5. Search Prefix
6. Search Suffix
7. Search Word
0. Exit""

def main():
    print("Text:\n" + TEXT)
    while True:
        print(MENU)
        choice = input("Enter choice: ").strip()

        if choice == "1":
            show("Dates", find(DATE, TEXT))
        elif choice == "2":
            show("Phone numbers", find(PHONE, TEXT))
        elif choice == "3":
            show("Hashtags", find(HASHTAG, TEXT))
        elif choice == "4":
            show("Mentions", find(MENTION, TEXT))
        elif choice == "5":
            kw = input("Enter prefix: ").strip()
            show(f"Words starting with '{kw}'", find(rf"\b{re.escape(kw)}\w*", TEXT))
        elif choice == "6":
            kw = input("Enter suffix: ").strip()
            show(f"Words ending with '{kw}'", find(rf"\w*{re.escape(kw)}\b", TEXT))
        elif choice == "7":
            kw = input("Enter word: ").strip()
            show(f"Word '{kw}'", find(rf"\b{re.escape(kw)}\b", TEXT))
        elif choice == "0":
            print("Goodbye!")
            break
        else:
            print("Invalid choice, try again.")

if __name__ == "__main__":
    main()

```

OUTPUT:

```
File Edit Selection View Go Run ... PYTHON VS
Problems Debug Console Terminal Output Ports Jupyter SQL Language Server

2. Search Phone Number
3. Search Hashtag
4. Search Mention
5. Search Prefix
6. Search Suffix
7. Search Word
0. Exit
Enter choice: 1

Dates: 1 match(es)
- 12/09/2026

===== TEXT SEARCH ENGINE =====
1. Search Date
2. Search Phone Number
3. Search Hashtag
4. Search Mention
5. Search Prefix
6. Search Suffix
7. Search Word
0. Exit
Enter choice: 2

Phone numbers: 1 match(es)
- 9876543210

===== TEXT SEARCH ENGINE =====
1. Search Date
2. Search Phone Number
3. Search Hashtag
4. Search Mention
5. Search Prefix
6. Search Suffix
7. Search Word
0. Exit

>< 0 0 0 Connect Spaces: 4 UTF-8 Python Python
```



Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Q. No. / Criteria	Problem Understanding	Algorithm	Code Implementation	Competitive Performance	Test Case Handling	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____